Shenzhen Hiwonder Technology Co,Ltd

Hiwonder

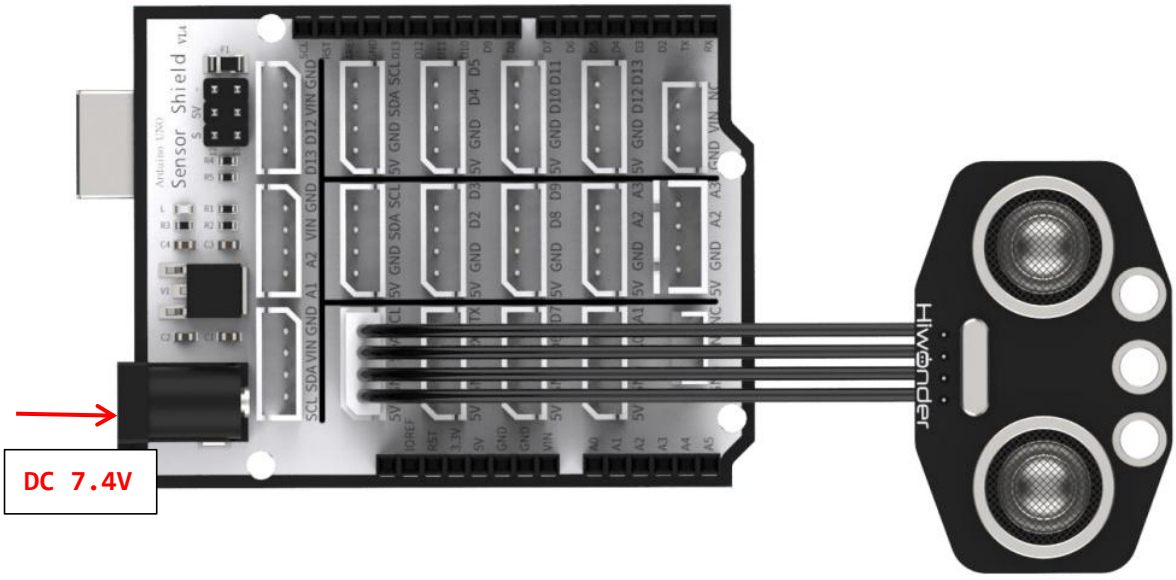
Arduino Development Tutorial

V1.0

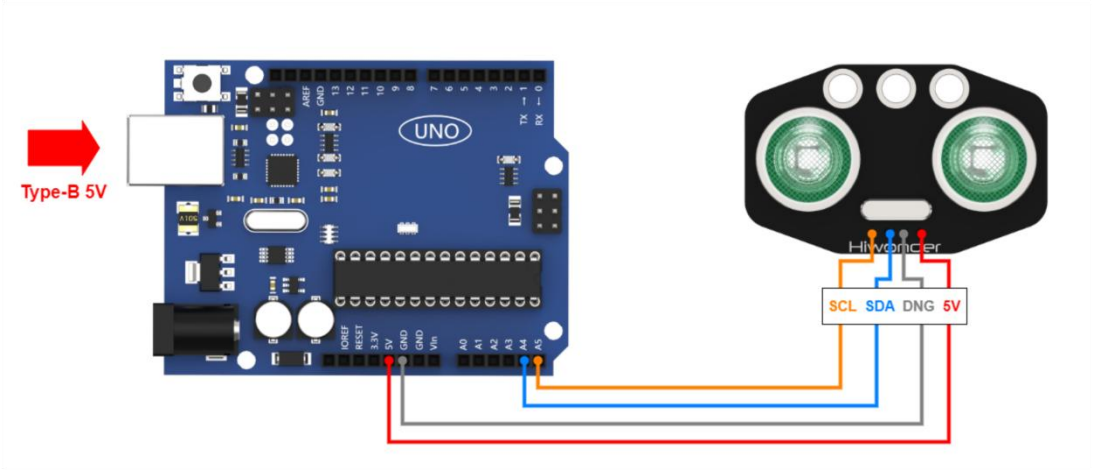
1. Getting Ready

1.1 Wiring Instruction

This section demonstrates connecting to pins A2 and A3 on the Arduino expansion board using a 4-pin cable. Refer to the following image:



If you don't have an Arduino expansion board, you can directly connect to the Arduino development board using DuPont wires. The connection diagram is shown below:



Note:

① If using Hiwonder lithium battery, please connect the red wire to the “+” terminal and the black wire to the “-” terminal of the DC connector.

② If the connector cable is not connected to the lithium battery, do not directly connect it to the battery connector cable to avoid a short circuit between the positive and negative terminals.

1.2 Environment Configuration

Install Arduino IDE into the computer, where the software package locates in “2. Software Tools & Program Collection\01 Installation Package of Each Software\Arduino Installation Package”. You can learn detailed usage instructions of Arduino IDE in the corresponding directory.

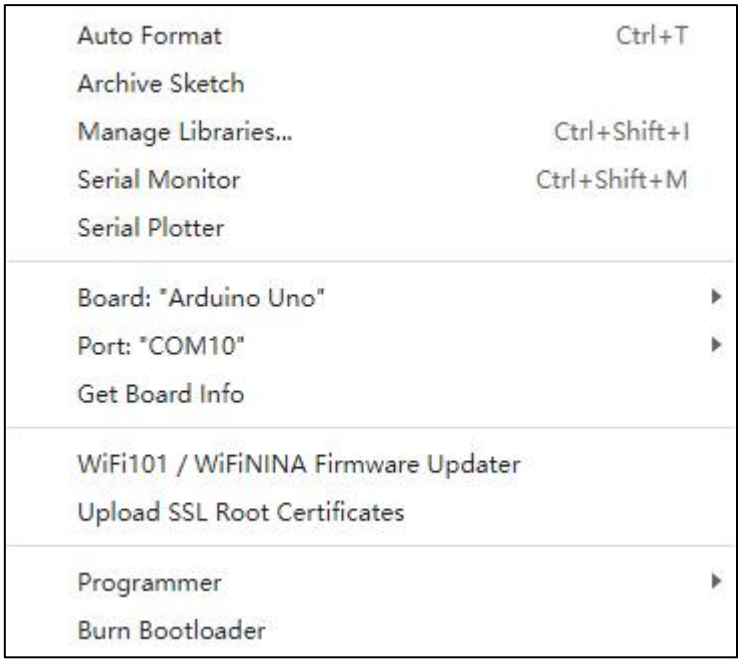
2. Test Case

This case displays the values detected by the glowing ultrasonic sensor through the terminal window, and emits different lights based on the detected values.

2.1 Program Download

1)Firstly, connect the Arduino UNO development board with the expansion board to the computer through USB cable. Then open Arduino IDE, click the “File->New” sequentially, and import the code from the same directory as this section.

2)In addition, select correct development board and port (the following ports are for example only. Please refer to the actual connections.), compile the program and upload it.



3)After the code is uploaded successfully, click to open the serial monitor. Then set the baud rate to 9600 and observe the phenomenon.

2.2 Program Outcome

Use the glowing ultrasonic sensor to test the distance of obstacles. According to the distance of the obstacle, two RGB lights are used to indicate whether the distance is too close or too far. When the distance is too short, the RGB glows with flashing red light; when the distance is moderate, the RGB lights up blue; and when the distance is too far, the RGB displays green.

```
COM12 (Arduino Uno)
Distance: 74 mm
Distance: 74 mm
Distance: 73 mm
Distance: 73 mm
Distance: 72 mm
Distance: 72 mm
Distance: 66 mm
Distance: 150 mm
Distance: 180 mm
Distance: 212 mm
Distance: 128 mm
Distance: 117 mm
Distance: 121 mm
Distance: 154 mm
Distance: 175 mm
Distance: 231 mm
Distance: 283 mm
Distance: 241 mm
Distance: 144 mm
Distance: 173 mm
Distance: 324 mm
Distance: 450 mm
```

2.3 Brief Analysis of Case Program

- **Import library file**

```
#include "Ultrasound.h"
```

Import ultrasonic library file, which is in the same path as the program. The import in double quotes means to import the file in the same path as the program.

- **Serial port initialization**

```
Ultrasound ultrasound;   //Instantiating ultrasound classes

void setup() {
  // put your setup code here, to run once:
  Serial.begin(9600);
}
```

Instantiate the ultrasonic library, and the baud rate of initializing ports is 9600.

- **Filter function**

```
#define FILTER_N 3                    //Recursive average filtering method
int filter_buf[FILTER_N + 1];
int Filter() {
  int i;
  int filter_sum = 0;
  filter_buf[FILTER_N] = ultrasound.GetDistance();      //Read ultrasonic measurements
  for(i = 0; i < FILTER_N; i++) {
    filter_buf[i] = filter_buf[i + 1];  //Move all data to the left and discard the lower ones
    filter_sum += filter_buf[i];
  }
  return (int)(filter_sum / FILTER_N);
}
```

Use the recursive averaging filtering method to discard the lower bits of the ultrasonic distance measurement value. This action can eliminate data bias caused by the instability of the sound wave.

- **Main function**

```
void loop() {
  // put your main code here, to run repeatedly:
  static uint32_t Time = 0;
  static uint8_t step = 0;
  int s;
  int distance = Filter();     //Obtain filter output value
  Serial.print("Distance: ");Serial.print(distance);Serial.println(" mm"); //Obtain and print distance

  if (distance > 0 && distance <= 80){
      ultrasound.Breathing(1, 0, 0, 1, 0, 0);        //breathy light mode, period 0.1 seconds, red
   }

  else if (distance > 80 && distance <= 180){
      s = map(distance, 80, 180, 0, 255);
      ultrasound.Color((255-s), 0, 0, (255-s), 0, 0);    //red gradient
   }
```

```
   else if (distance > 180 && distance <= 320){
      s = map(distance, 180, 320, 0, 255);
      ultrasound.Color(0, 0, s, 0, 0, s);              //blue gradient
   }

   else if (distance > 320 && distance <= 500){
      s = map(distance, 320, 500, 0, 255);
      ultrasound.Color(0, s, 255-s, 0, s, 255-s);            //green gradient
   }
  else if (distance > 500){
      ultrasound.Color(0, 255, 0, 0, 255, 0);          //green
   }
   delay(300);
}
```

In the main program, store the filtered ultrasonic detection value in the “distance” variable, and print the distance value through serial port. At the same time, determine whether the distance is less than or equal to 80mm. If it is, the distance value is determined as too close, then set the RGB to breathing light mode with a period of 0.1s and red color. The effect is shown as flashing red light.

After that, if the distance is between 80 to 180, the RGB light will be a red gradient; if the distance is between 180 to 320, the RGB light will be a blue gradient; if the distance is between 320 to 500, the RGB light will be a green gradient; if the distance is greater than 500, the RGB light will be a constant green.